

ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное
учреждение высшего образования
**«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»**

Факультет компьютерных наук
Образовательная программа «Компьютерные науки и анализ данных»

ОТЧЁТ О КОМАНДНОМ ПРОГРАММНОМ ПРОЕКТЕ

на тему:

**LLM Web Parsing — сервис автоматизированной разметки веб-страниц на
основе больших языковых моделей
(итоговый отчёт)**

Выполнили студенты группы #КНАД242, 2 курс:

Широкая Анна Антоновна

Беляев Даниил Андреевич

Копейкин Михаил Александрович

Принял руководитель проекта:

Козловский Виктор Евгеньевич

Старший разработчик, X5 Tech

Москва, 2026

Аннотация

В отчёте представлены итоговые результаты разработки AI-сервиса автоматизированной разметки веб-страниц LLM Web Parsing. Проект реализует постановку задачи Parsmart: по образцу страницы товара сервис автоматически подбирает CSS-селекторы для всех её ключевых полей (название, цена, артикул, изображение) и позволяет переиспользовать эти селекторы на тысячах страниц того же домена без повторного обращения к языковой модели. Ключевая ценность решения — универсальность: оно работает с любым e-commerce сайтом без необходимости программировать отдельные правила извлечения под каждый магазин.

Разработанная система применяет большие языковые модели Llama 3.1 8B Instruct и GPT-4o-mini для семантического понимания HTML-разметки. Это позволяет автоматически идентифицировать расположение ключевых элементов даже на ранее не встречавшихся страницах. На итоговом этапе по требованию заказчика реализована функция интерактивных действий — система не только читает страницу, но и взаимодействует с ней: нажимает кнопки, добавляет товары в корзину, переходит между карточками. Поиск интерактивных элементов выполняется через эвристическую систему скоринга (с поддержкой русского, английского, французского и немецкого языков) с автоматическим LLM-fallback при низкой уверенности эвристики.

Архитектура реализована на FastAPI с поддержкой асинхронной обработки, валидацией через Pydantic, асинхронным доступом к базе данных SQLite через aiosqlite, интеграцией LangChain и Playwright. Покрывание тестами — 139 автоматизированных тестов в 10 модулях. Система контейнеризирована через Docker и доступна в публичном репозитории на GitHub.

Ключевые слова

веб-парсинг, большие языковые модели, LLM, автоматизированная разметка, CSS-селекторы, TOON, Hugging Face, Llama, GPT-4o-mini, Playwright, FastAPI, browser automation,

интерактивные действия, e-commerce, интернет-магазины, headless Chromium, антибот-защита, playwright-stealth, эвристический скоринг, LangChain, Pydantic.

Содержание

Для построения содержания после открытия файла в Microsoft Word выполните: «Ссылки» → «Оглавление» → «Автособираемое оглавление». Все заголовки в документе размечены стилями Heading 1 / Heading 2 / Heading 3 и будут собраны автоматически.

1. Введение

1.1. Описание проекта

LLM Web Parsing представляет собой полнофункциональный AI-сервис автоматизированного извлечения данных и взаимодействия с веб-страницами интернет-магазинов. Сервис носит рабочее название «Parsmart» по постановке задачи курса. В отличие от классических парсеров, требующих программирования отдельных правил извлечения под каждый сайт, разработанное решение использует семантическое понимание HTML-структуры большими языковыми моделями, что обеспечивает универсальность и устойчивость к редизайнам.

Система поддерживает три режима работы. В режиме прямого извлечения LLM анализирует очищенный HTML и возвращает структурированный JSON с полями товара. В режиме CSS-селекторов LLM один раз генерирует CSS-селекторы для каждого поля карточки товара, после чего система переиспользует эти селекторы на тысячах страниц того же домена без повторного обращения к языковой модели. В режиме интерактивных действий (реализован по требованию заказчика на финальном этапе) система открывает страницу в headless-браузере и выполняет действия: нажимает кнопки, добавляет товары в корзину, переходит между карточками.

1.2. Актуальность и значимость

Актуальность разработки обусловлена двумя факторами. Во-первых, фундаментальными ограничениями традиционных методов веб-парсинга. Классические инструменты типа Scrapy или комбинации BeautifulSoup и requests требуют создания отдельного набора CSS-селекторов или XPath-выражений для каждого целевого сайта. Любое изменение дизайна, переименование классов или реструктуризация HTML приводит к поломке парсера и необходимости его доработки вручную. По оценкам, до 40% рабочего времени инженеров, занимающихся скрейпингом данных, тратится именно на поддержку существующих парсеров.

Во-вторых, растущими потребностями e-commerce и аналитического бизнеса в автоматизированном сборе данных. Компании, занимающиеся ценовой аналитикой, мониторингом конкурентов, агрегацией ассортимента маркетплейсов, обрабатывают сотни тысяч страниц ежедневно. Применение больших языковых моделей радикально меняет парадигму веб-парсинга: LLM понимают семантику HTML-разметки, идентифицируют общие паттерны представления товаров и работают на любом сайте без явного программирования правил.

Дополнительный пласт актуальности связан с тем, что современные интернет-магазины активно используют интерактивные элементы — кнопки «В корзину», «Купить сейчас», динамические выпадающие меню. Парсеры, ограниченные только чтением статического HTML, не способны автоматически взаимодействовать с такими сайтами. Реализованная функция интерактивных действий закрывает эту нишу, открывая возможности для автоматизированного тестирования интернет-магазинов, имитации действий пользователя для маркетинговых исследований, интеграции с внешними сервисами и QA-системами.

1.3. Цель проекта

Целью проекта является разработка и доведение до работоспособного состояния автоматизированной системы извлечения данных с веб-сайтов и взаимодействия с ними, основанной на больших языковых моделях. Система должна обеспечивать универсальность работы с произвольными интернет-магазинами без перепрограммирования логики под каждый сайт, высокую точность извлечения структурированных данных, способность выполнять интерактивные действия и предоставлять программный интерфейс для интеграции с внешними сервисами.

1.4. Задачи проекта

В рамках проекта решались следующие задачи:

Проведение анализа предметной области веб-парсинга и существующих технологических решений (включая Scrapy, Octoparse, ParseHub, Diffbot, ScrapingBee, Apify).

Проектирование архитектуры системы на основе FastAPI с асинхронной обработкой и определение технологического стека.

Реализация программного пайплайна загрузки, очистки и извлечения данных с веб-страниц с использованием LLM.

Реализация режима генерации и применения CSS-селекторов в формате TOON, совместимом с заданием Parsmart.

По требованию заказчика — реализация модуля интерактивных действий на странице с универсальным эвристическим поиском кнопок.

Обеспечение асинхронной обработки и персистентного хранения извлечённых данных через aioredis.

Покрывание системы автоматизированными тестами (unit + integration), включая мокирование внешних вызовов Playwright и LLM API.

Проведение тестирования и валидация результатов на синтетических фикстурах и реальных интернет-магазинах, включая крупные российские маркетплейсы.

Документирование архитектуры и публикация проекта в открытом репозитории на GitHub.

2. Обзор аналогов

В рамках первого этапа работы был проведён детальный анализ существующих решений в области автоматизированного веб-парсинга. Аналоги классифицированы на три группы по принципу работы: классические парсеры по жёстким правилам, no-code платформы с визуальным конструктором, AI-сервисы на основе машинного обучения и больших языковых моделей.

2.1. Классические парсеры по жёстким правилам

Scrapy

Scrapy — наиболее популярный open-source фреймворк для веб-скрейпинга на Python. Обладает зрелой экосистемой, асинхронной архитектурой на Twisted, встроенной поддержкой пайплайнов обработки, middleware, экспорта в JSON/CSV. Имеет обширное сообщество и тысячи готовых интеграций.

Основные ограничения Scrapy: требует явного описания XPath или CSS-селекторов для каждого сайта; при изменении дизайна парсер ломается; не справляется со страницами на React и Vue без отдельной интеграции со Splash или Playwright. По сравнению с разработанным решением Scrapy требует ручного программирования правил под каждый сайт, в то время как LLM Web Parsing подбирает их автоматически.

BeautifulSoup и requests

Не фреймворк, а библиотека для парсинга HTML, часто используемая в связке с HTTP-клиентом requests или httpx. Лёгкая, простая в освоении, гибкая. Однако работает только со статическим HTML и требует написания низкоуровневой логики извлечения для каждого сайта вручную. В разработанном решении BeautifulSoup используется как инструмент очистки HTML и применения уже сгенерированных LLM селекторов, но не является ядром логики.

2.2. No-code платформы

Octoparse

Десктопное приложение со встроенным браузером для визуального построения парсеров. Пользователь кликает по элементам страницы, и платформа автоматически предлагает CSS-селекторы. Поддерживает облачное выполнение, имеет API.

Ограничения: проприетарное, бесплатный тариф ограничен по числу запросов и страниц; закрытый исходный код; при смене дизайна сайта пользователь вручную перестраивает шаги парсера; нет автоматической миграции на новый макет. LLM Web Parsing превосходит Octoparse по ключевому критерию автоматизации: селекторы переопределяются одним обращением к LLM, без ручного клика по элементам. Также конфиг хранится в открытом TOON-формате и может быть отредактирован вручную или передан другому инструменту.

ParseHub

Конкурент Octoparse с аналогичным UX. Имеет те же сильные и слабые стороны. Дополнительным недостатком является отсутствие полноценного self-hosted режима — все парсеры выполняются на облачной инфраструктуре ParseHub.

Apify

Облачная платформа с маркетплейсом готовых «акторов» для парсинга популярных сайтов. Преимущество: десятки тысяч готовых решений с хорошим качеством скрейпинга, прокси-серверы из коробки.

Недостатки: платный сервис с моделью оплаты за compute units; закрытая экосистема, не позволяющая запускать акторы локально или в собственном облаке. LLM Web Parsing является self-hosted решением без vendor lock-in.

2.3. AI/LLM-решения

Diffbot

Коммерческий сервис, использующий *computer vision* и машинное обучение для автоматического извлечения данных. Преимущества: очень высокое качество извлечения, поддержка различных типов контента (товары, статьи, видео, профили), большая база уже размеченных сайтов.

Недостатки: платный сервис с минимальным тарифом 299\$/месяц; проприетарное решение; не возвращает пользователю CSS-селекторы — только готовые JSON-результаты, что неудобно для downstream-обработки или интеграции в собственный парсер. LLM Web Parsing возвращает именно CSS-селекторы, которые могут быть использованы вне сервиса, например в Scrapy. Стоимость эксплуатации значительно ниже — основные расходы это бесплатный HuggingFace Inference API или копейки за GPT-4o-mini OpenAI.

ScrapingBee

Облачный *browser-as-a-service* для парсинга с возможностью передать промпт и получить JSON. Простой API, встроенная инфраструктура прокси-серверов и решение капч. Недостатки: платный; AI-извлечение реализовано как простая передача промпта без сохранения сгенерированных селекторов. Каждый запрос — это новый вызов LLM, что значительно дороже подхода LLM Web Parsing с кэшированием.

2.4. Сводное сравнение

Проведённый анализ показывает, что ни одно из изученных решений не сочетает одновременно следующие характеристики: использование LLM для автоматической разметки; возврат самих CSS-селекторов, позволяющий переиспользовать их вне сервиса; open-source и self-hosted развёртывание; поддержку SPA через headless-браузер; способность к интерактивным действиям на странице. Разработанная система LLM Web Parsing закрывает именно эту нишу. Кэширование сгенерированных селекторов обеспечивает скорость и стоимость на уровне классических парсеров, а автоматическая генерация и интерактивное взаимодействие со страницей — гибкость и устойчивость к редизайну, типичные для AI-решений.

3. Архитектура системы

3.1. Высокоуровневая схема

Архитектура системы построена по многоуровневому принципу обработки веб-контента с применением современных инструментов автоматизации. Процесс начинается с получения URL целевой страницы через REST API на FastAPI. Playwright загружает страницу в headless-браузере Chromium с применением stealth-патчей для обхода базовых антибот-систем. BeautifulSoup4 выполняет предварительную очистку HTML, сокращая его размер в 10–20 раз. LangChain интегрирует языковую модель с возможностью переключения между провайдерами (HuggingFace или OpenAI). Pydantic валидирует результаты, aiosqlite обеспечивает асинхронный персистентный доступ к базе данных SQLite.

В режиме CSS-селекторов LLM вызывается один раз для опорной страницы домена и возвращает структуру селекторов. Сгенерированная разметка проходит автоматическую валидацию: каждый селектор проверяется на исходной странице через BeautifulSoup, и невалидные или ничего-не-нашедшие селекторы сбрасываются в null с записью пояснения в специальное поле комментариев. После валидации разметка сохраняется в SQLite и используется для всех последующих страниц того же домена без обращения к LLM.

В режиме интерактивных действий система открывает страницу в браузере, выполняет эвристический поиск нужного элемента по тексту, aria-label, data-атрибутам, class-паттернам и role. При недостаточной уверенности эвристики (менее 30 баллов в системе скоринга) выполняется fallback на LLM-запрос: модель возвращает CSS-селектор вместе с уровнем уверенности. После идентификации элемента выполняется клик с подробным логированием каждого шага.

3.2. Технологический стек

Технологический стек проекта основан на Python 3.12:

Backend: FastAPI 0.115 с Uvicorn в качестве ASGI-сервера. Обеспечивает асинхронную обработку, автоматическую генерацию OpenAPI-документации на /docs и интеграцию с Pydantic для валидации.

Валидация данных: Pydantic 2.9 с строгими схемами и кастомными валидаторами для цены, артикула, URL изображений.

База данных: SQLite через aiosqlite 0.20 для асинхронного доступа. Три таблицы: parsed_products, domain_markups, interaction_runs.

LLM-интеграция: LangChain 0.3 + huggingface-hub 1.16 с двухуровневым fallback на прямой Inference API. Опционально OpenAI GPT-4o-mini для production.

Парсинг страниц: Playwright 1.47 с headless Chromium и stealth-патчами через playwright-stealth 2.0.

Предобработка HTML: BeautifulSoup4 4.12 с lxml 5.3.

Тестирование: pytest 8.3 + pytest-asyncio + httpx для интеграционных тестов FastAPI.

Деплой: Docker + docker-compose для контейнерного запуска одной командой.

Конфигурация: python-dotenv для управления переменными окружения через .env-файл.

3.3. Принципы проектирования

Архитектура построена на нескольких ключевых принципах. Чёткое разделение ответственности по слоям: каждый модуль решает ровно одну задачу — `fetcher` загружает HTML через `Playwright`, `cleaner` очищает его, `llm.parser` обращается к языковой модели, `applier` применяет CSS-селекторы через `BeautifulSoup`, `models.schemas` содержит Pydantic-валидацию, `database.db` реализует CRUD-операции над SQLite, `toon.serializer` формирует выходной формат, `api.routes` оркеструет всю логику через HTTP-эндпоинты.

Lazy initialization для LLM-клиента: фактическое соединение с HuggingFace или OpenAI устанавливается только при первом запросе, не при импорте модуля. Это позволяет создавать LLMParser с тестовым ключом-заглушкой для модульных тестов и подменять внутренние компоненты через `unittest.mock` без попытки реального обращения к внешнему API.

Двухуровневый *fallback* в LLM-слое. `LangChain` активно меняет API между минорными версиями, что регулярно приводит к ошибкам стандартной цепочки. Реализован *fallback*: при любом исключении стандартной цепочки `template | llm` выполняется прямой вызов `huggingface_hub.InferenceClient.chat_completion()`. Это обеспечивает работоспособность даже при поломках совместимости `LangChain`.

Толерантный JSON-парсер. Языковые модели регулярно возвращают JSON с *trailing comma*, в *markdown-обёртке*, с одинарными кавычками вместо двойных, с пояснительным текстом до и после, с Python-литералами `None/True/False`. Реализованный парсер сначала пробует `json.loads` напрямую, затем итеративно чинит каждую из перечисленных проблем.

Кэширование через CSS-селекторы. Главное архитектурное решение — кэшировать именно CSS-селекторы, а не сырые результаты парсинга. Преимущества: один вызов LLM покрывает любое число страниц одного домена; при изменении содержимого товара (новая цена, изменение скидки) повторный вызов LLM не требуется — селектор найдёт новое значение; при редизайне сайта достаточно повторно вызвать генерацию селекторов; TOON-конфиг можно передать другим инструментам, поскольку формат открытый.

4. Вклад участников команды

4.1. Широкая Анна

4.1.1. Роль в проекте

Моя роль в проекте заключалась в составлении плана работ, координации взаимодействия в команде и анализе предметной области. На начальном этапе я провела комплексный анализ существующих решений веб-парсинга, исследовала их сильные и слабые стороны, определила архитектурные принципы построения системы. На финальном этапе провела детальный сравнительный анализ разработанной системы с коммерческими аналогами `Diffbot`, `ScrapingBee`, `Apify`, `Octoparse`.

Анализ показал, что традиционные парсеры требуют создания отдельных правил извлечения для каждого сайта, что делает процесс разработки трудоёмким. Более критичная проблема — хрупкость решений при изменении дизайна сайтов. Я идентифицировала применение больших языковых моделей как перспективное решение, позволяющее обойти эти ограничения. LLM обладают способностью понимать семантику HTML-разметки и идентифицировать паттерны представления товаров, что даёт универсальность без программирования правил под каждый магазин.

Мною была разработана архитектурная концепция системы с многоуровневой обработкой данных. Архитектура включает REST API на FastAPI для приёма запросов, модуль загрузки страниц через Playwright с *stealth-патчами*, препроцессор HTML на BeautifulSoup4, LLM-интеграцию через LangChain, систему валидации и нормализации через Pydantic, асинхронное хранилище на `aiosqlite` SQLite. Каждый компонент решает строго определённую задачу, что обеспечивает модульность и упрощает тестирование.

Был составлен детальный технологический стек. Python 3.12 выбран благодаря зрелой экосистеме библиотек для веб-технологий, машинного обучения и работы с данными. HuggingFace Inference API обеспечивает доступ к Llama 3.1 8B Instruct бесплатно, а OpenAI GPT-4o-mini добавлен как опциональный *production-провайдер* с более высоким качеством.

Координация работы команды включала еженедельные синхронизации прогресса, распределение задач с учётом компетенций участников и контроль выполнения. На финальном этапе я обеспечила корректную интеграцию модуля интерактивных действий в существующую архитектуру, переработанную после требований заказчика.

4.1.2. Выявленные проблемы и решения

Основной проблемой на этапе планирования стал выбор оптимальной языковой модели. Коммерческие модели OpenAI обеспечивают высокое качество, но требуют финансовых

затрат. Открытые модели через Hugging Face бесплатны, но имеют ограничения по rate-limit и качеству. В качестве решения была реализована двухпровайдерная архитектура с автоматическим выбором: при наличии OPENAI_API_KEY система использует GPT-4o-mini, иначе — Llama 3.1 8B Instruct через HuggingFace.

Второй сложностью стало определение допустимого размера HTML для передачи в языковую модель. Современные интернет-магазины генерируют объёмные HTML-документы, превышающие контекстное окно большинства бесплатных моделей. Реализован адаптивный preprocessor с агрессивной очисткой и smart-truncation, сохраняющий критичные блоки документа.

Третья проблема — обеспечение отказоустойчивости. Реализован механизм retry с экспоненциальной задержкой через tenacity, система таймаутов на всех этапах, безопасная обработка исключений с возвратом структурированного status='error' вместо падения с 500-й ошибкой.

4.1.3. Дальнейшее развитие

На дальнейших этапах планируется расширение тестового датасета до 200–300 страниц с обеспечением репрезентативности по категориям. Необходимо разработать систему мониторинга качества в реальном времени, автоматически детектирующую деградацию извлечения и запускающую регенерацию селекторов при изменении дизайна сайта. Также перспективно внедрение системы A/B тестирования различных версий промптов на production-трафике с автоматическим выбором лучшей.

4.2. Копейкин Михаил

4.2.1. Роль в проекте

Моя ответственность заключалась в подготовке данных, декомпозиции бизнес-процесса парсинга и реализации программного пайплайна обработки веб-контента. Разработал и протестировал основные модули загрузки, очистки и интеграции с языковой моделью. На финальном этапе реализовал модуль интерактивных действий по требованию заказчика.

Первым реализованным компонентом стал REST API на FastAPI для приёма запросов на парсинг. FastAPI выбран благодаря асинхронной природе, высокой производительности при обработке множественных запросов, автоматической генерации OpenAPI-документации, интеграции с Pydantic. Эндпоинт /api/parse принимает URL целевой страницы и опционально источник, выполняет всю цепочку обработки и возвращает структурированный JSON с извлечёнными данными о товаре.

Ключевым модулем стал fetcher, реализующий загрузку веб-страниц через Playwright в headless-режиме Chromium. В отличие от простых HTTP-клиентов, Playwright запускает полноценный браузер, что позволяет корректно обрабатывать динамический контент JavaScript-фреймворков React, Vue и Angular. Метод использует событие networkidle для гарантии завершения сетевых запросов и базовую паузу для отложенных скриптов.

Для повышения шансов обхода антибот-защит крупных маркетплейсов интегрирован пакет playwright-stealth, патчающий навигаторские признаки автоматизации: navigator.webdriver, эмуляция Chrome runtime, реалистичный WebGL fingerprint. Добавлен полный набор реалистичных HTTP-заголовков Chrome 121 на macOS включая Sec-Ch-Ua, Sec-Fetch-* и Accept-Language с локалью ru-RU.

Реализован препроцессор HTML на базе BeautifulSoup4. Модуль cleaner выполняет агрессивную очистку: удаляет теги script, style, noscript, iframe, svg, meta, link, head, footer, nav. Прореживает атрибуты, оставляя только class, src, href, alt, itemprop. Итеративно удаляет пустые контейнеры. Для интерактивного режима введён специальный флаг keep_interactive=True, при котором button, input, form и атрибуты aria-label, role, data-action бережно сохраняются, поскольку именно на них основано распознавание кнопок.

Реализована LLM-интеграция через LangChain с двухуровневым fallback. На уровне один используется цепочка template | llm стандартного LangChain. При любой ошибке выполняется прямой вызов huggingface_hub.InferenceClient.chat_completion() — это обеспечивает работоспособность даже при поломках совместимости LangChain. Температура установлена в 0.0 для детерминированности, максимальное количество токенов ограничено 1024.

На финальном этапе реализован модуль interactor для интерактивных действий. Поддерживаемые действия: add_to_cart (добавить в корзину), buy_now (купить сейчас), open_product (открыть карточку), click_text (клик по тексту), custom_selector (клик по явному CSS). Алгоритм эвристического поиска кликабельных элементов (element_finder) разработан Даниилом — я отвечал за интеграцию его модуля с Playwright и за саму логику клика.

Логика поиска элемента трёхэтапная: сначала эвристический скоринг через element_finder.find_best(html, intent), при низкой уверенности — LLM-fallback (новый промпт INTERACTIVE_ELEMENT_TEMPLATE в LangChain-цепочке), при недостатке уверенности обоих — корректный возврат status=not_found без ложных срабатываний. После клика

страница ожидает `networkidle`, затем считывается `title` и сравнивается с пред-кликом для подтверждения что действие выполнено. Все шаги сохраняются в `InteractionResult.log` с `timestamp_ms` и записываются в БД.

4.2.2. Выявленные проблемы и решения

Основной проблемой стала обработка динамического контента на JavaScript-фреймворках. Некоторые сайты используют `React` или `Vue.js` с ленивой загрузкой компонентов, и `Playwright` иногда завершал загрузку до полной отрисовки. Решением стало двухступенчатое ожидание: сначала `domcontentloaded` (быстро и гарантированно), затем попытка `networkidle` с тайм-аутом без блокировки. Опционально добавлены параметры `wait_for_selector` и `extra_wait_ms` для совсем медленных SPA.

Сложностью оказались антибот-системы. `Cloudflare`, `DataDome` и аналогичные механизмы детектируют автоматизированные браузеры. Для базового обхода интегрированы `playwright-stealth`, ротация `User-Agent`, реалистичные HTTP-заголовки и `timezone Europe/Moscow`. Это позволило пройти антибот `Wildberries` и `Citilink`, однако продвинутые защиты `Ozon` (`Antibot Captcha` от `DataDome`) и `DNS-shop` (HTTP 403 от `Akamai` на основе `TLS-fingerprint`) требуют резидентных прокси и выходят за рамки self-hosted решения.

Парсинг JSON из ответов LLM оказался нестабильным. Модели добавляют пояснительный текст, используют одинарные кавычки или допускают `trailing commas`. Реализован толерантный парсер `_extract_json`, последовательно применяющий правила нормализации: удаление markdown-обёрток, замена одинарных кавычек, удаление `trailing comma`, замена Python-литералов `None/True/False` на JSON-эквиваленты.

4.2.3. Дальнейшее развитие

Планируется оптимизация производительности пайплайна через параллельную обработку множественных URL одного домена с переиспользованием `browser-context`. Перспективен переход к PostgreSQL для production-нагрузок, добавление Redis-кэша результатов парсинга и фоновой обработки через `Celery`. Также планируется расширение поддерживаемых действий: `multistep flows` (например, выбор размера → добавление в корзину), авторизация перед действием, скриншоты до и после клика.

4.3. Беляев Даниил

4.3.1. Роль в проекте

Моя зона ответственности в проекте охватывает три направления: разработку модуля CSS-селекторов с экспортом в формат TOON (это центральный модуль системы по постановке задачи `ParSMART`), реализацию слоя данных (Pydantic-схемы, асинхронный доступ к SQLite, CRUD-операции) и обеспечение качества (методология тестирования, разработка автотестов, end-to-end проверки на реальных интернет-магазинах). Дополнительно на финальном этапе я разработал модуль эвристического поиска интерактивных элементов (`element_finder`), который интегрирован Михаилом в модуль `interactor`.

4.3.2. Модуль CSS-селекторов и формат TOON

Центральная задача проекта по постановке `ParSMART` — генерация CSS-селекторов для типовой страницы товара и их переиспользование на всех страницах того же домена. Я реализовал три модуля: `selectors.py` (генерация и валидация селекторов), `applier.py` (применение селекторов к HTML без обращения к LLM), `toon/serializer.py` (сериализация в формат TOON, совместимый с примером `domains.toon` из задания).

Логика генерации работает следующим образом. Опорная страница товара загружается через `fetcher`, очищается через `cleaner`, и передаётся в специализированный промпт `SELECTORS_GENERATION_TEMPLATE`. Я разработал текст промпта таким образом, чтобы LLM возвращал стабильный JSON фиксированной структуры с шестью полями: `name`, `price`, `currency`, `pictures`, `category`, `article`. В промпт встроены явные инструкции: предпочитать класс- и атрибут-селекторы вместо глубоких descendant-цепочек; избегать auto-generated хэшей вида `css-1a2b3c`; использовать только реально присутствующие в HTML классы. Без этих инструкций модели регулярно «галлюциновали», выдумывая несуществующие классы.

Сгенерированные селекторы проходят автоматическую валидацию в функции `validate_selectors`. Каждый селектор применяется к исходному HTML через `BeautifulSoup`. Если селектор синтаксически некорректен или ничего не находит — он сбрасывается в `null`, а в поле `markdownComments` разметки добавляется текстовое пояснение. Это критически важный защитный слой от галлюцинаций LLM: даже если модель придумала несуществующий класс, в финальной разметке его не будет, и пользователь увидит честное предупреждение.

Модуль `applier` применяет сохранённые селекторы к любым URL того же домена без обращения к LLM. Это даёт скорость классических парсеров при универсальности AI: один вызов модели покрывает тысячи страниц одного магазина. Я реализовал интеллектуальную нормализацию извлечённых значений. Функция `_normalize_price` обрабатывает множество форматов: «1 999 Р» → 1999.0, «1 999,99 Р» → 1999.99, европейский формат «1.999,99 Р» →

1999.99. Функция `_normalize_article` склеивает пробелы внутри числовых последовательностей («2006 4875» → «20064875») при этом сохраняет alphanumeric SKU как есть («SM-A55», «LEN-001»). Функция `_absolutize` превращает относительные URL изображений в абсолютные, обрабатывая три случая: уже абсолютный URL, `protocol-relative` (`//cdn.x.com/...`), относительный путь.

Сериализатор TOON я разработал с нуля, поскольку формат самописный и стандартных библиотек для него не существует. Формат отличается от YAML двумя особенностями: массивы декларируются с указанием длины (`markdownComments[3]`: вместо `markdownComments`), элементы массива нумеруются (1: «текст»). Реализованы `dump_domains` для сериализации списка разметок, вспомогательные функции для `productPage`, `cartPage`, `orderPage`, корректное экранирование двойных кавычек внутри значений. Порядок полей `productPage` стабильно фиксирован: `category`, `currenсу`, `name`, `pictures`, `price`, `article` — точно как в эталонном `domains.toon` из задания. Это важно для совместимости и проверки соответствия результата работы системы исходному ТЗ.

4.3.3. Слой данных: Pydantic и SQLite

Я отвечаю за всю работу с данными в системе. На уровне схем разработал семейство Pydantic-моделей в файле `app/models/schemas.py`: `ParseRequest` (входящий запрос с валидацией URL по regex и ограничением длины), `ProductData` (структура извлечённых данных с custom-валидаторами на цену и URL изображения), `DomainMarkup` и `ProductPageSelectors` (полная разметка домена в TOON-формате), `CartPageSelectors` и `OrderPageSelectors` (опциональные структуры для корзины и оформления заказа), `SelectorsGenerateRequest` и `SelectorsApplyRequest` (для CSS-режима), `InteractRequest` и `InteractResponse` (для интерактивных действий). Все модели имеют strict-валидацию: некорректные данные приводят к `422 Unprocessable Entity`, что улучшает диагностику ошибок.

Особое внимание уделено custom-валидаторам. Валидатор цены отфильтровывает аномальные значения: отрицательная цена → `ValidationError`, цена выше 10 миллионов → `ValidationError`, значение округляется до двух знаков после запятой. Валидатор URL изображения проверяет схему `http(s)://` и возвращает `None` для некорректных значений (вместо падения). Валидатор названия обрезает строки длиннее 500 символов и превращает пустые строки в `None`. Эти правила я выработал на основе анализа реальных ошибок LLM при обработке российских интернет-магазинов.

Слой персистентного хранения я реализовал на SQLite через `aiosqlite` — асинхронный драйвер совместимый с архитектурой FastAPI. Спроектирована схема из трёх таблиц. Таблица `parsed_products` хранит историю всех запросов прямого извлечения (`url`, `source`, `status`, `title`, `price`, `article`, `image_url`, `raw_response`, `created_at`). Таблица `domain_markup` хранит сгенерированные CSS-разметки в виде JSON-блоба с уникальным ключом по нормализованному домену и UPSERT-логикой при повторной генерации. Таблица `interaction_runs` хранит журнал интерактивных действий с подробным техническим логом каждого шага. Созданы индексы по `url` и `created_at` для ускорения типичных запросов.

Реализован полный набор CRUD-операций в `app/database/db.py`. Функция `save_domain_markup` использует SQLite UPSERT через `ON CONFLICT` для атомарного обновления разметки. Функция `list_domain_markup` возвращает все разметки отсортированными по времени обновления. Все операции асинхронные через `context manager async with aiosqlite.connect()`, что обеспечивает корректное закрытие соединений даже при исключениях.

4.3.4. Эвристический поиск элементов (модернизация)

На финальном этапе проекта по требованию заказчика реализован модуль интерактивных действий. Моя зона ответственности в этом модуле — разработка алгоритма эвристического поиска кликабельных элементов (`app/scrapper/element_finder.py`). Михаил отвечал за интеграцию с Playwright и логику собственно клика, я — за интеллектуальный подбор CSS-селектора.

Разработана система скоринга, ранжирующая каждый потенциально-кликабельный элемент (`button`, `a`, `input[type=submit]`, `div` с `role=button` или `onclick`) по семи признакам. Точное совпадение текста с известной фразой намерения даёт 20 баллов, содержит фразу — 15, все слова присутствуют — 8. `aria-label` учитывается с коэффициентом 0.9 от текста, `title` — с коэффициентом 0.7. Класс или `id`, соответствующий регулярному выражению (например, `/add[-_]?to[-_]??(cart|bag|basket)/`), даёт 25 баллов. Атрибут `role=button` — 5 баллов. Тип `meza button` — 10, `input[type=submit]` — 8, `a` — 4. Anti-pattern'ы («в избранное», «wishlist», «compare») снижают балл на 50. Disabled-кнопки снижают балл на 30. Конкретные веса я подбирал экспериментально на разнообразных примерах, чтобы `scoring` корректно работал на русских и английских интерфейсах.

Особое внимание уделит многоязычности. В словаре `ADD_TO_CART_TEXTS` представлены формулировки на четырёх языках: русский («в корзину», «добавить в корзину», «купить», «заказать», «положить в корзину»), английский («add to cart», «add to bag», «buy now»), французский («ajouter au panier»), немецкий («in den warenkorb»). Аналогичные словари для intent'ов `buy_now` и `open_product`. Добавление нового языка — буквально одна строка в словаре, без изменений в основной логике.

Разработана функция `build_selector`, строящая уникальный CSS-селектор для найденного элемента с учётом приоритетов: сначала пытаемся использовать `id` (если не `auto-generated` хэш), затем `data-testid`, `data-action`, `input[name=...]`, затем уникальную комбинацию классов, в крайнем случае — путь от ближайшего предка с осмысленным `id`. Особый блок логики — функция `_looks_autogenerated`, распознающая хэш-классы CSS-in-JS (`css-1a2b3c`), `styled-components` (`sc-aBcD1`), `Next.js JSX` (`jsx-1234567890`), `Material-UI` (`MuiButton-root-12345`). Такие классы пропускаются, поскольку они меняются между сборками сайта и приводят к нестабильности селекторов.

4.3.5. Тестирование

Помимо разработки кода я отвечал за обеспечение качества всей системы. Спроектирована трёхуровневая стратегия тестирования. На уровне `unit` покрываются отдельные функции без сети и базы данных. На уровне `integration` используется реальный экземпляр `FastAPI` приложения и временная `SQLite`-база, но внешние вызовы `Playwright` и `LLM API` мокируются через `unittest.mock`. На уровне `end-to-end` выполняется ручной прогон через работающий сервер на синтетических фикстурах и реальных интернет-магазинах.

Разработано 139 автоматизированных тестов в 10 модулях. Лично я написал тесты, относящиеся к моей кодовой зоне: `test_schemas.py` (16 тестов `Pydantic`-валидации), `test_applier.py` (19 тестов применения CSS-селекторов и нормализации цены/артикула), `test_toon.py` (12 тестов сериализатора с проверкой порядка полей и экранирования), `test_database.py` (8 тестов `CRUD SQLite`), `test_selectors_api.py` (9 тестов CSS-эндпоинтов с моками), `test_element_finder.py` (20 тестов эвристики на многоязычных примерах). Тесты `test_cleaner.py`, `test_llm_parser.py`, `test_api.py` и `test_interact_api.py` написаны Михаилом совместно со мной — я отвечал за `code review` и валидацию покрытия.

Проведены `end-to-end` тесты на синтетической фикстуре `sample_product.html` с эталонными значениями из `T3 Parsmart`, на реальном сайте `books.toscrape.com` с двумя разными книгами для демонстрации переиспользования селекторов без `LLM`, на демонстрационном `demoblaze.com`, на сайтах `automationexercise.com` и `scrapingclub.com`. На завершающем этапе проведены прогоны на крупных российских маркетплейсах: `Wildberries`, `Ozon`, `DNS-shop`, `M.Видео`, `Citilink`. Все результаты задокументированы в файлах `docs/testing.md` и `docs/testing_results.md` с подробными логами каждого шага.

Ключевая демонстрация качества: на `books.toscrape.com` сгенерированные селекторы корректно применились к двум разным книгам без повторного вызова `LLM` («*A Light in the Attic*» — £51.77, «*Tipping the Velvet*» — £53.74). На `Wildberries` система успешно прошла антибот-защиту: интерактивное действие `add_to_cart` было выполнено корректно — `LLM` подобрал селектор `.product-card__add-basket.j-add-to-basket` с `confidence 90`, клик отработал.

4.3.6. Выявленные проблемы и решения

При разработке генератора CSS-селекторов основной проблемой стали галлюцинации языковой модели. На сложных страницах `LLM` иногда придумывала несуществующие классы или возвращала синтаксически невалидный CSS. Решением стала автоматическая валидация: каждый сгенерированный селектор сразу проверяется на исходной `HTML`-странице через `BeautifulSoup`. Невалидные селекторы сбрасываются в `null` с записью текстового пояснения в поле `markdownComments`. Это превратило недостаток (галлюцинации `LLM`) в преимущество (прозрачную систему обратной связи для пользователя).

При проектировании `Pydantic`-моделей сложным оказался баланс между строгостью валидации и пропуском «грязных» данных. Слишком жёсткие правила приводили к тому, что валидные ответы `LLM` отбрасывались (например, если `LLM` возвращал цену с пробелом). Слишком мягкие — пропускали явный мусор. Решением стали `custom`-валидаторы с `pre-обработкой`: например, валидатор цены сначала нормализует строку (убирает пробелы, символы валюты, заменяет запятую на точку), а потом уже проверяет числовое значение. Это позволило быть толерантным к форматам ввода при сохранении семантической строгости результата.

При разработке эвристики поиска кнопок основной сложностью стало правильное балансирование весов. Я перебрал десятки вариантов на тестовом наборе кнопок с реальных сайтов, прежде чем нашёл стабильный режим: точное совпадение текста (20) важнее `class-паттернов` (25), но `class-паттерны` достаточно сильны чтобы перекрыть `тег-бонус`. `Anti-`

pattern'ы (-50) полностью отменяют положительные баллы, что критично для отсеечения «избранного».

При e2e-тестировании на маркетплейсах сложностью оказалась субъективность *ground truth*. Для некоторых страниц неочевидно, какое именно название считать правильным — краткое из *Н1* или расширенное с дополнительными характеристиками. Также LLM-модели иногда выдают разные результаты при повторных запросах даже при температуре 0.0. Решением стало введение *confidence score* для маркировки результатов, требующих ручной проверки, и *chunking* тестового датасета на детерминированные (синтетические фикстуры) и стохастические (реальные сайты) компоненты.

4.3.7. Дальнейшее развитие

По линии CSS-селекторов планируется добавление поддержки *cartPage* и *orderPage* в генератор (сейчас они только в схеме, но LLM-промпт пока работает только для *productPage*). Это требует разработки специализированных промптов для каждого типа страницы и логики автоматической классификации (определить, что текущая страница — карточка товара, корзина или *checkout*).

По линии слоя данных планируется миграция с *SQLite* на *PostgreSQL* для *production*-нагрузок. Интерфейс модуля *app/database/db.py* построен абстрактно, миграция требует только замены драйвера (*aiosqlite* → *asyncpg*) и адаптации SQL-диалекта. Также перспективно добавление *Redis*-кэша для горячих разметок и истории парсинга.

По линии эвристики поиска планируется расширение поддерживаемых интенгов: *leave_review* (оставить отзыв), *select_option* (выбор размера/цвета), *apply_filter* (применение фильтра в каталоге), *subscribe* (подписка на *email*). Каждый новый *intent* — это запись в словарь *INTENTS* из пяти строк, никаких изменений в основном коде.

По линии тестирования планируется внедрение системы *benchmark*-тестирования для сравнительного анализа различных языковых моделей на едином датасете: *Llama 3.1 8B/70B*, *Mistral 7B*, *GPT-4o-mini/GPT-4o*, *Claude 3.5 Haiku*. Также эксперименты с промпт-инженерией: длина инструкций, *few-shot learning* с примерами успешного извлечения, специализированные промпты для конкретных маркетплейсов. Перспективна разработка системы автоматической детекции изменений в дизайне сайтов с триггерной регенерацией селекторов и *post-processing* эвристик (фильтрация нереалистичных цен, валидация форматов артикулов).

5. Реализация ключевых компонентов

5.1. REST API на FastAPI

Backend реализован на *FastAPI* с поддержкой асинхронной обработки. Эндпоинты разделены на четыре группы. Первая группа — прямое извлечение: *POST /api/parse* запускает обработку, *GET /api/results* возвращает историю с пагинацией, *GET /api/results/{id}* — конкретную запись, *DELETE /api/results* очищает историю. Вторая группа — CSS-селекторы по постановке *Parsmart*: *POST /api/selectors/generate* генерирует селекторы по опорной странице, *POST /api/selectors/apply* применяет их к новому URL без обращения к LLM, *GET /api/selectors* возвращает список сохранённых разметок, *GET /api/selectors/export.toon* выгружает их в формате *TOON*, совместимом с примером *domains.toon* из задания.

Третья группа — интерактивные действия: *POST /api/interact* универсальная ручка для любого действия, *POST /api/cart/add* шорткат для добавления в корзину, *GET /api/interactions/runs* история всех интерактивных запусков, *GET /api/interactions/runs/{id}* детали с полным техническим логом. Четвёртая группа — служебные: *GET /health* для *health-check*, *POST /api/interactions* для записи пользовательских действий.

5.2. Очистка HTML

Типичная страница интернет-магазина содержит 200–500 килобайт сырого *HTML*, что многократно превышает контекстное окно бесплатных моделей. Модуль *HTMLCleaner* выполняет агрессивную очистку и компактификацию документа до 12 тысяч символов в режиме извлечения и до 20 тысяч в интерактивном режиме.

Алгоритм очистки состоит из четырёх шагов. На первом удаляются теги *script*, *style*, *noscript*, *iframe*, *svg*, *meta*, *link*, *head*, *footer*, *nav*, *aside*, *header*, *form* (последний — только в режиме извлечения, в интерактивном режиме формы сохраняются). На втором прореживаются атрибуты: для извлечения остаются *src*, *href*, *alt*, *class*, *data-article*, *data-sku*, *id*; для интерактивного режима дополнительно сохраняются *aria-label*, *role*, *type*, *name*, *value*, *data-action*, *data-event*, *data-testid*, *onclick*, *disabled*. На третьем итеративно удаляются пустые контейнеры. На четвёртом выполняется сжатие пробельных символов и адаптивное усечение.

5.3. Режим CSS-селекторов с экспортом в TOON

Главное архитектурное решение проекта — кэшировать CSS-селекторы, а не сырые данные парсинга. В этом режиме LLM вызывается только один раз на домен: при обработке опорной страницы товара модель возвращает JSON с CSS-селекторами для шести полей: *name*, *price*, *currency*, *pictures*, *category*, *article*.

Селекторы проходят автоматическую валидацию на исходной странице. Если селектор синтаксически некорректен или ничего не находит, он сбрасывается в `null`, а в поле `markdownComments` разметки добавляется соответствующее предупреждение. Это критически важно: языковые модели иногда галлюцинируют, придумывая несуществующие CSS-классы. Валидация защищает от таких ошибок.

Валидированная разметка сохраняется в `SQLite` через `UPSERT` по ключу домена. Дальнейшая обработка любых URL того же домена выполняется без вызова LLM: модуль `applier` применяет сохранённые селекторы через `BeautifulSoup`, нормализует цену (удаление пробелов, символов валюты, нормализация запятой и точки), нормализует артикул (склейка пробелов внутри числовой последовательности), абсолютизирует относительные URL изображений. Это даёт скорость классических парсеров при универсальности AI.

Сохранённые разметки могут быть выгружены через `GET /api/selectors/export.toon` в формате `TOON`, совместимом с примером `domains.toon` из задания `Parsmart`. Серуализатор поддерживает все конструкции формата: нумерованные массивы `domains[N]` и `markdownComments[N]`, вложенные объекты `productPage`, `cartPage`, `orderPage`; стабильный порядок полей; корректное экранирование кавычек.

5.4. Интерактивные действия

Реализация модуля интерактивных действий — ответ на финальное требование заказчика. Цель: система должна не только читать страницу, но и взаимодействовать с ней — нажимать кнопки, добавлять товары в корзину. Критическое требование универсальности: решение должно работать с разными интернет-магазинами без переписывания кода под каждый сайт.

Для выполнения этих требований реализован модуль `element_finder` с системой эвристического скоринга. Каждый потенциально-кликабельный элемент (`button`, `a`, `input[type=submit]`, `div` с `role=button` или `onclick`) получает баллы по нескольким признакам. Точное совпадение текста с известной фразой даёт 20 баллов, содержит фразу — 15, все слова присутствуют — 8. Класс или `id`, соответствующий регулярному выражению (например, `add-to-cart`, `buy-now`), даёт 25 баллов. `role=button` — 5. Тег `button` — 10, `input[type=submit]` — 8, `a` — 4. `Anti-pattern`'ы (`избранное`, `wishlist`) снижают балл на 50. `Disabled`-кнопки снижают на 30. Глубокая вложенность — на 3.

Многоязычность реализована через словарь фраз для каждого намерения. Для `add_to_cart` поддерживаются «в корзину», «купить», «заказать», «add to cart», «buy now», «purchase», «ajouter au panier», «in den warenkorb», «agregar al carrito» и десятки других. Добавление нового языка — одна строка в словаре.

Минимальный порог уверенности эвристики — 30 баллов. Если `score` ниже, выполняется `LLM-fallback`: модель получает очищенный HTML и возвращает JSON с предложенным CSS-селектором, уровнем `confidence` от 0 до 100 и текстовым обоснованием. Если LLM возвращает `confidence` ниже 50 или `None` — система корректно возвращает `status=not_found` без попыток случайного клика. Это критически важно: лучше честно сообщить о ненайденном элементе, чем кликнуть не туда.

Сборка надёжного CSS-селектора (функция `build_selector`) учитывает приоритеты: `id` (если не `auto-generated` хэш) → `data-testid` → `data-action` → `input[name]` → `tag` с осмысленными классами → `path` от ближайшего предка с `id`. Игнорируются хэш-классы `CSS-in-JS` (`css-1a2b3c`), `styled-components` (`sc-aBcD1`), `Next.js` (`jsx-1234567890`), `Material-UI` (`MuiButton-root-12345`).

Сам клик выполняется через `Playwright` с подробным логированием каждого шага: `goto`, `page_loaded`, `heuristic`, `candidate` (с `score`), `llm` (если был `fallback`), `click`, `post_click_wait`, `done`. Все шаги с `timestamp_ms` сохраняются в БД в поле `log_json` для последующего анализа.

6. Тестирование

6.1. Методология

Применена трёхуровневая стратегия тестирования. На уровне `unit` покрываются отдельные функции без сети и базы данных. На уровне `integration` используется реальный экземпляр `FastAPI` приложения и временная `SQLite`-база, но внешние вызовы `Playwright` и LLM API мокируются через `unittest.mock`. На уровне `end-to-end` выполняется ручной прогон через работающий сервер на синтетических фикстурах и реальных интернет-магазинах.

Всего реализовано 139 автоматизированных тестов в 10 модулях. Время полного прогона на одной машине — менее одной секунды, поскольку внешние API замокированы. Это позволяет запускать тесты в CI/CD-пайплайне без подключения к сети.

6.2. Распределение тестов по модулям

`test_schemas.py` — 16 тестов `Rydantic`-валидации: URL должен начинаться с `http(s)`, не быть пустым, не превышать 2048 символов; цена положительная и реалистичная; пустое название превращается в `None`.

test_cleaner.py — 11 тестов очистки HTML: *script/style* удаляются, *img src* сохраняется, пустые контейнеры удаляются, HTML усекается при превышении лимита.

test_llm_parser.py — 12 тестов: *_extract_json* парсит чистый JSON, JSON в markdown, с trailing comma, с null-полями; *_normalize* обрабатывает «1 990₽» → 1990.0 и альтернативные имена полей; *extract_product_data* корректно использует LangChain и fallback.

test_applier.py — 19 тестов: *validate_selectors* отбрасывает несуществующие селекторы; *apply_selectors* извлекает поля и абсолютизирует URL; *_normalize_price* парсит разные форматы цен; *_normalize_article* корректно работает с SKU и числовыми артикулами.

test_toon.py — 12 тестов сериализатора: пустой список → *domains[0]*; null-селекторы не попадают в выход; комментарии нумеруются с 1; кавычки экранируются; порядок полей *productPage* стабилен (*category*, *currency*, *name*, *pictures*, *price*, *article*).

test_database.py — 8 тестов CRUD: *save* и *get parsed_products*, *save error* без полей, *count*, *history*, *raw_response* сохраняется.

test_api.py — 13 тестов *endpoints*: *health*, *results empty*, *parse* с моками, *parse fetch error*, *results not found*, *interactions*, *clear*.

test_selectors_api.py — 9 тестов CSS-режима: *invalid URL* → 422, *success* с моками, фильтрация галлюцинированных селекторов, *apply* без LLM, *list*, *export.toon*.

test_element_finder.py — 20 тестов эвристики: русские/английские/французские/немецкие кнопки, *anti-patterns*, *build_selector* с приоритетами, *_looks_autogenerated* для CSS-in-JS.

test_interact_api.py — 9 тестов интерактивных *endpoints*: *invalid URL* и *action* → 422, *success*, *not found*, *error* возвращается со статусом 200 (не 5xx), шорткат */cart/add*, история *runs*.

6.3. End-to-end тестирование на локальной фикстуре

Подготовлены две HTML-фикстуры. Первая — *sample_product.html* — повторяет структуру эталонной страницы товара из задания Parsmart с полями *name*, *price*, *article*, *image*, *breadcrumbs*. Вторая — *product_with_buttons.html* — содержит четыре разных кнопки: «В корзину», «В избранное» (*anti-pattern*), «Купить сейчас», «Закончился» (*disabled*). Это позволяет проверить корректность *scoring system* на типичных сценариях.

Результаты на *sample_product.html*. Прямое извлечение через LLM: *title* «Электрогриль Tefal OptiGrill Elite GC750D30» — совпадает с эталоном; *article* «2006 4875» — найден; *image_url* — корректный. Генерация CSS-селекторов: *name* «*m-product .main .name*» точно совпадает с эталоном ТЗ Parsmart; *pictures* «*#m-zoomable-image picture img*» точно совпадает с эталоном; *article* корректен. Применение селекторов в режиме *apply* (без LLM): артикул автоматически нормализован «2006 4875» → «20064875».

Результаты на *product_with_buttons.html*. Действие *add_to_cart*: эвристика нашла *#addToCartBtn* со *score* 55.0; LLM не вызывался (*score* выше порога); клик выполнен успешно; *title* страницы изменился с «Кофемашина Bosch ТКА8633 — Sample Shop» на «Кофемашина Bosch ТКА8633 — добавлено в корзину», что подтверждает реальное выполнение JavaScript-обработчика.

6.4. End-to-end тестирование на реальных сайтах

Проведены прогоны на нескольких категориях реальных сайтов: демонстрационные тестовые магазины, реальные русские маркетплейсы, сайты с разной степенью антибот-защиты.

6.4.1. books.toscrrape.com (демо-сайт Scrapy)

На странице книги «A Light in the Attic» все три режима отработали успешно. Прямое извлечение: *title* корректно, *article* (UPC-код) корректно, *image_url* корректно. Генерация селекторов вернула все шесть полей без предупреждений валидатора: *name*=«*.product_page h1*», *price*=«*.product_main .price_color*», *article*=«*.table-striped tbody tr:nth-child(1) td*», *pictures*=«*#product_gallery .carousel-inner .item*», *category*=«*.breadcrumb li.active*». Применение этих же селекторов к ДРУГОЙ книге того же домена («Tipping the Velvet») без повторного вызова LLM корректно извлекло данные новой книги: *title*=«Tipping the Velvet», *price*=53.74, *article*=«90fa61229261140a». Это ключевая демонстрация концепции Parsmart: один вызов LLM покрывает тысячи страниц одного домена.

6.4.2. Wildberries

На крупном российском маркетплейсе Wildberries система успешно прошла антибот-защиту благодаря интеграции *playwright-stealth* и реалистичных HTTP-заголовков. На странице товара */catalog/192222089* (тройник PP-R с внутренней резьбой) интерактивное действие *add_to_cart* выполнено корректно. Эвристика дала *score* 29 (ниже порога), система автоматически переключилась на LLM-fallback. LLM подобрал селектор «*.product-card__add-basket.j-add-to-basket*» с *confidence* 90 и обоснованием «This element has a clear class name indicating it is the add-to-basket button». Клик выполнен за 16 секунд, без ошибок.

Это демонстрация работы двухуровневой стратегии: эвристика → LLM-fallback. На реальной странице Wildberries DOM содержит десятки кандидатов (включая блок «Все 68 предложений от 70 120 ₽» с альтернативными продавцами), но LLM правильно идентифицировал именно главную кнопку добавления в корзину.

6.4.3. demoblaze.com

Демонстрационный e-commerce сайт, специально созданный для тестирования автоматизации. Прогон `add_to_cart`: эвристика обнаружила кандидата с score 24 (ниже порога), сработал LLM-fallback. GPT-4o-mini вернул селектор `«.product-content .btn.btn-success»` с confidence 90. Клик выполнен успешно.

6.4.4. Известные ограничения

Зафиксированы границы применимости системы. На Ozon stealth не проходит — возвращается «Antibot Captcha» от DataDome. На DNS-shop возвращается HTTP 403 от Akamai на основании анализа TLS-fingerprint браузера. Это известные ограничения headless-парсинга без резидентных прокси-сетей, общие для всех self-hosted решений. Согласуется с разделом обзора аналогов: даже коммерческие сервисы Diffbot и Apify арендуют резидентные прокси именно для обхода такой защиты.

На M.Video stealth работает (страница загружается), но контент карточки товара загружается JavaScript-ом из закрытого API через несколько секунд после события `networkidle`, и наш HTML-snapshot получается до полной отрисовки. Это решается параметром `extra_wait_ms` (5–10 секунд), но снижает общую производительность.

На `books.toscrape.com` странице книги система корректно возвращает `status=not_found` на запрос `add_to_cart` — на этой странице действительно нет кнопки добавления в корзину (она появляется только на странице каталога). И эвристика, и LLM согласились, что элемента нет. Это важный показатель качества: система не «галлюцинирует», а честно сообщает о неудаче.

7. Заключение

В рамках курсового проекта разработана полнофункциональная система LLM Web Parsing для автоматизированной разметки веб-страниц и интерактивного взаимодействия с ними. Реализованы все ключевые модули, заданные в постановке Parsmart, плюс модернизация по требованию заказчика на финальном этапе.

Достигнуты следующие результаты. Реализован REST API на FastAPI с автоматической генерацией OpenAPI-документации (Swagger UI на `/docs`). Реализован веб-загрузчик на Playwright с stealth-патчами и retry-логикой через `tenacity`, успешно проходящий антибот-защиту Wildberries и Citilink. Реализован препроцессор HTML на BeautifulSoup4 с адаптивным усечением до 12–20 тысяч символов. Реализован LLM-парсер с интеграцией LangChain и двухуровневым fallback на прямой HuggingFace API, поддерживающий Llama 3.1 8B Instruct и GPT-4o-mini с автоматическим выбором провайдера. Реализован модуль генерации и применения CSS-селекторов в формате TOON, совместимом с примером `domains.toon` из задания. Реализован сериализатор TOON-формата. Реализована система валидации через Pydantic. Реализован асинхронный слой персистентного хранения на SQLite через `aiosqlite` с тремя таблицами: `parsed_products`, `domain_markups`, `interaction_runs`.

По требованию заказчика реализован модуль интерактивных действий с универсальным эвристическим поиском кнопок (поддержка четырёх языков: русский, английский, французский, немецкий), системой `scoring` и LLM-fallback. Поддерживаются пять действий: `add_to_cart`, `buy_now`, `open_product`, `click_text`, `custom_selector`. Все действия сохраняются в базу данных с подробным техническим логом каждого шага.

Веб-интерфейс реализован на ванильном JavaScript с четырьмя вкладками: прямое извлечение, CSS-селекторы, действия, история. Покрытие тестами — 139 автоматизированных тестов в 10 модулях, время полного прогона менее одной секунды. Замокированы внешние вызовы Playwright и LLM API, что делает тесты быстрыми, детерминированными и пригодными для CI/CD. Система контейнеризирована через Docker и `docker-compose` для запуска одной командой.

Проект опубликован в открытом репозитории на GitHub по адресу `github.com/ItsBelyaev/llmweb-parser`. Доступны две ветки: `main` с основной реализацией и `feature/interactive-actions`, содержащая модернизацию мая 2026 года и слитая в `main` после успешного тестирования.

Гибкая интеграция с различными провайдерами языковых моделей (Hugging Face и OpenAI) позволяет выбирать оптимальный баланс между качеством, производительностью и стоимостью обработки в зависимости от конкретного сценария использования. Применение CSS-селекторов вместо повторных вызовов LLM на каждой странице обеспечивает скорость промышленных парсеров при универсальности AI-решения. Эвристический поиск кнопок с LLM-fallback обеспечивает работу на любом интернет-магазине без программирования правил под конкретный сайт, что прямо соответствует ключевому требованию технического задания.

Перспективы развития проекта включают расширение тестового датасета до 200–300 страниц, разработку системы автоматической детекции редизайнов с триггерной регенерацией селекторов, реализацию *multistep flows* для интерактивных действий, миграцию на PostgreSQL и Redis для *production*-нагрузок, добавление Celery для фоновой обработки длинных пайплайнов, реализацию системы A/B тестирования различных версий промптов на *production*-трафике.

8. Список литературы

- Brown T. et al. *Language Models are Few-Shot Learners* // *Advances in Neural Information Processing Systems*. — 2020. — Vol. 33. — pp. 1877–1901.
- Liu P. et al. *Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing* // *ACM Computing Surveys*. — 2023. — Vol. 55, No. 9. — pp. 1–35.
- Touvron H. et al. *Llama 2: Open Foundation and Fine-Tuned Chat Models* // arXiv:2307.09288. — 2023.
- OpenAI. *GPT-4 Technical Report* // arXiv:2303.08774. — 2023.
- Ramírez S. *FastAPI Documentation*. — Tiangolo, 2024. — URL: <https://fastapi.tiangolo.com/>
- Microsoft Corporation. *Playwright Documentation*. — 2024. — URL: <https://playwright.dev/python/>
- Beautiful Soup Documentation. Crummy Software. — 2024. — URL: <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>
- Pydantic v2 Documentation. Pydantic Services Inc. — 2024. — URL: <https://docs.pydantic.dev/>
- LangChain Documentation. LangChain Inc. — 2024. — URL: <https://python.langchain.com/>
- Hugging Face Inference API Documentation. Hugging Face Inc. — 2024. — URL: <https://huggingface.co/docs/api-inference/>
- Mitchell T.M., Cohen W.W. *Never-Ending Learning* // *Communications of the ACM*. — 2018. — Vol. 61, No. 5. — pp. 103–115.
- SQLAlchemy and aiosqlite Documentation. — 2024. — URL: <https://docs.sqlalchemy.org/>, <https://aiosqlite.omnilib.dev/>
- Diffbot AI-powered Web Data Extraction. — 2024. — URL: <https://www.diffbot.com/>
- ScrapingBee Web Scraping API. — 2024. — URL: <https://www.scrapingbee.com/>
- Apify Web Scraping Platform Documentation. — 2024. — URL: <https://docs.apify.com/>
- Octoparse Visual Web Scraping. — 2024. — URL: <https://www.octoparse.com/>
- Scrapy 2.11 Documentation. — Scrapinghub Ltd, 2024. — URL: <https://docs.scrapy.org/>
- Playwright Stealth Plugin (Python). — Mattwmaster58, 2024. — URL: https://github.com/Mattwmaster58/playwright_stealth
- Постановка задачи Parsmart 2025. — Курсовой проект студентов НИУ ВШЭ, 2025.
- ITESCIA Tech. *DataDome and Antibot Defense Mechanisms*. — Technical Report, 2023.